

Absolutely. Below is an **Answer Bank for Units I–III**, based on the uploaded *Analysis of Algorithms – Classroom Teaching Script*. I have kept the answers suitable for **university examination preparation**, with concise 2-mark answers and more detailed 5/10-mark answers. The source covers Units I–III as Introduction, Divide and Conquer, and Greedy Method respectively.

AOA_Teaching_Script +2

ANALYSIS OF ALGORITHMS

Question Bank with Answers – Units I to III

UNIT I – INTRODUCTION

Part A – 2 Marks Questions with Answers

Q1. Define Time Complexity.

Answer:

Time complexity is the amount of computational work performed by an algorithm as a function of the input size n . It describes how the running time grows when the input size increases.

Example:

- Linear search $\rightarrow O(n)$
 - Nested loop $\rightarrow O(n^2)$
-

Q2. Define Space Complexity.

Answer:

Space complexity is the amount of memory required by an algorithm during its execution as a function of input size.

It includes:

- Input storage
 - Variables
 - Arrays
 - Temporary memory
 - Auxiliary data structures
-

Q3. What is Growth Rate?

Answer:

Growth rate describes how the running time or memory requirement of an algorithm increases as the input size n increases.

Common growth rates are:

$$O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), O(2^n)$$

Q4. Why are constants ignored in asymptotic analysis?

Answer:

Constants are ignored because asymptotic analysis focuses on the growth rate for large input sizes.

For example:

$$5n^2 + 10n + 20$$

is represented as:

$$O(n^2)$$

because n^2 is the dominant term.

Q5. Differentiate between Time Complexity and Space Complexity.

Time Complexity

Space Complexity

Measures computational time/work

Measures memory requirement

Depends on number of operations

Depends on memory used

Example: $O(n^2)$

Example: $O(n)$

Q6. Define Big-O notation.

Answer:

Big-O notation gives an **upper bound** on the growth rate of a function.

Formally,

$$f(n) = O(g(n))$$

if there exist positive constants c and n_0 such that:

$$f(n) \leq cg(n)$$

for all:

$$n \geq n_0$$

Q7. Define Omega notation.

Answer:

Omega (Ω) notation gives a **lower bound** on the growth rate.

$$f(n) = \Omega(g(n))$$

if:

$$f(n) \geq cg(n)$$

for all $n \geq n_0$, where $c > 0$.

Q8. Define Theta notation.

Answer:

Theta (Θ) notation provides both an upper and lower bound.

$$f(n) = \Theta(g(n))$$

if there exist positive constants c_1, c_2, n_0 such that:

$$c_1g(n) \leq f(n) \leq c_2g(n)$$

for all $n \geq n_0$.

Q9. What is the difference between Big-O and Theta?

Answer:

- **Big-O:** Upper bound.
- **Theta:** Tight bound.

For example:

$$3n^2 + 5n + 2 = \Theta(n^2)$$

and therefore it is also:

$$O(n^2)$$

Q10. What is the Master Method?

Answer:

The Master Method is a technique for solving recurrence relations of the form:

$$T(n) = aT(n/b) + f(n)$$

It compares $f(n)$ with:

$$n^{\log_b a}$$

to determine the complexity.

AOA_Teaching_Script

Q11. State the three cases of the Master Method.

Answer:

For:

$$T(n) = aT(n/b) + f(n)$$

let:

$$p = n^{\log_b a}$$

Then:

1. **Case 1:** $f(n)$ is smaller than p
2. **Case 2:** $f(n)$ and p have the same order
3. **Case 3:** $f(n)$ is larger than p

For Case 2, the typical result is:

$$\Theta(n^{\log_b a} \log n)$$

Q12. What is a recurrence relation?

Answer:

A recurrence relation defines the running time of a recursive algorithm in terms of the

running time of smaller inputs.

Example:

$$T(n) = 2T(n/2) + n$$

This is the recurrence for Merge Sort.

Q13. State the formula for the sum of first n natural numbers.

Answer:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Q14. Why is mathematical induction useful in algorithm analysis?

Answer:

Mathematical induction helps prove that a formula or property is true for **all values of n** rather than checking only a few values. The notes specifically use induction to prove the summation formula.

AOA_Teaching_Script

Q15. What is the complexity of Selection Sort?

Answer:

Best case:

$$\Theta(n^2)$$

Average case:

$$\Theta(n^2)$$

Worst case:

$$\Theta(n^2)$$

Selection Sort performs nearly the same number of comparisons regardless of input ordering.

AOA_Teaching_Script

Q16. What is the best-case complexity of Insertion Sort?

Answer:

$$\Theta(n)$$

It occurs when the array is already or nearly sorted.

AOA_Teaching_Script

Q17. What is the worst-case complexity of Insertion Sort?

Answer:

$$\Theta(n^2)$$

It occurs when the input is in reverse order.

Q18. What are P and NP?

Answer:

- **P:** Problems that can be solved efficiently in polynomial time.
- **NP:** Problems for which a proposed solution can be verified efficiently.

The notes use Sudoku as an intuitive example: checking a completed Sudoku is much easier than solving an empty one.

AOA_Teaching_Script

Q19. What is an NP-Complete problem?

Answer:

An NP-Complete problem is a problem that is both:

1. In NP, and
2. At least as hard as every problem in NP.

Examples mentioned in the notes include **SAT** and **Subset Sum**.

AOA_Teaching_Script

Q20. What is an NP-Hard problem?

Answer:

An NP-Hard problem is at least as hard as NP-Complete problems. It does not necessarily have to belong to NP.

For example, the optimization version of TSP is NP-Hard.

AOA_Teaching_Script

Part B – 5 Marks Questions with Answers

Q1. Explain Time and Space Complexity.

Answer:

When analyzing an algorithm, we mainly consider:

1. Time Complexity

It measures the amount of computational work performed as input size increases.

For example:

```
for i = 1 to n  
    print(i)
```

The loop executes n times.

Therefore:

$$T(n) = O(n)$$

2. Space Complexity

It measures the memory required by the algorithm.

For example:

```
int A[n];
```

requires memory proportional to n .

Therefore:

$$S(n) = O(n)$$

Algorithm analysis is generally independent of the particular machine or programming language because asymptotic analysis focuses on growth rate rather than exact execution time.

AOA_Teaching_Script

Q2. Explain Big-O, Ω and Θ .

Answer:

Big-O

Represents an upper bound.

$$f(n) = O(g(n))$$

Example:

$$3n^2 + 5n + 2 = O(n^2)$$

Omega

Represents a lower bound.

$$f(n) = \Omega(g(n))$$

Theta

Represents a tight bound.

$$f(n) = \Theta(g(n))$$

For:

$$f(n) = 3n^2 + 5n + 2$$

we can say:

$$f(n) = \Theta(n^2)$$

The notes provide a formal proof using $c = 10, n_0 = 1$.

AOA_Teaching_Script

Q3. Explain Selection Sort.

Answer:

Selection Sort repeatedly finds the smallest element from the unsorted portion and places it at the beginning.

Algorithm

```
SelectionSort(A, n)
  for i = 0 to n-2
    min = i

    for j = i+1 to n-1
      if A[j] < A[min]
        min = j

    swap(A[i], A[min])
```

Example

Consider:

[29, 10, 14, 37, 13]

Pass 1: Minimum = 10

[10, 29, 14, 37, 13]

Pass 2: Minimum = 13

[10, 13, 14, 37, 29]

Pass 3: Minimum = 14

[10, 13, 14, 37, 29]

Pass 4: Minimum = 29

[10, 13, 14, 29, 37]

Complexity

Best:

$$O(n^2)$$

Average:

$$O(n^2)$$

Worst:

$$O(n^2)$$

The number of comparisons is:

$$\begin{aligned} & (n - 1) + (n - 2) + \cdots + 1 \\ &= \frac{n(n - 1)}{2} \end{aligned}$$

Q4. Explain Insertion Sort.

Answer:

Insertion Sort builds the sorted array one element at a time.

Algorithm

```
InsertionSort(A, n)
  for i = 1 to n-1
    key = A[i]
    j = i-1

    while j >= 0 and A[j] > key
      A[j+1] = A[j]
      j = j-1

    A[j+1] = key
```

Example

Consider:

[37, 29, 14, 13, 10]

After inserting 29:

[29, 37, 14, 13, 10]

After inserting 14:

[14, 29, 37, 13, 10]

After inserting 13:

[13, 14, 29, 37, 10]

After inserting 10:

[10, 13, 14, 29, 37]

Complexity

Best case:

$$\Theta(n)$$

Worst case:

$$\Theta(n^2)$$

Insertion Sort is particularly useful for small or nearly sorted data.

AOA_Teaching_Script

Q5. Solve $T(n) = 2T(n/2) + n$.

Answer:

Given:

$$T(n) = 2T(n/2) + n$$

Compare with:

$$T(n) = aT(n/b) + f(n)$$

Therefore:

$$a = 2, \quad b = 2, \quad f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Thus:

$$f(n) = \Theta(n)$$

This is **Case 2** of Master Method.

Therefore:

$$T(n) = \Theta(n \log n)$$

This is the recurrence used for Merge Sort in the notes.

AOA_Teaching_Script

Q6. Solve $T(n) = 8T(n/2) + n^2$.

Answer:

$$a = 8, \quad b = 2, \quad f(n) = n^2$$

Calculate:

$$n^{\log_2 8} = n^3$$

Compare:

$$f(n) = n^2$$

with:

$$n^3$$

Since n^2 grows slower than n^3 , this is **Case 1**.

Therefore:

$$T(n) = \Theta(n^3)$$

The notes use this recurrence as an example of Case 1.

AOA_Teaching_Script

UNIT II – DIVIDE AND CONQUER

Part A – 2 Marks Questions with Answers

Q1. What is Divide and Conquer?

Answer:

Divide and Conquer is an algorithm design technique that divides a problem into smaller subproblems, solves the subproblems recursively, and combines their solutions.

Its three steps are:

1. Divide
2. Conquer
3. Combine

Q2. Give examples of Divide and Conquer algorithms.

Answer:

- Merge Sort
- Quick Sort
- Binary Search
- Min-Max algorithm
- Strassen's Matrix Multiplication

Q3. What is Merge Sort?

Answer:

Merge Sort is a Divide and Conquer sorting algorithm. It recursively divides an array into two halves, sorts each half and then merges the sorted halves.

Its complexity is:

$$\Theta(n \log n)$$

Q4. Why does Merge Sort require extra space?

Answer:

During merging, a temporary array is generally required to store the combined sorted result.

Therefore, Merge Sort requires:

$$O(n)$$

auxiliary space.

AOA_Teaching_Script

Q5. What is a pivot in Quick Sort?

Answer:

A pivot is an element selected to divide the array into two parts:

- Elements smaller than the pivot
- Elements greater than the pivot

The pivot is then placed in its correct position.

Q6. What is the worst-case complexity of Quick Sort?

Answer:

$$O(n^2)$$

It generally occurs when partitioning is highly unbalanced, such as when a poor pivot is repeatedly selected for already sorted input.

AOA_Teaching_Script

Q7. What is Binary Search?

Answer:

Binary Search is a searching algorithm that repeatedly divides a **sorted** search space into two halves.

Its time complexity is:

$$O(\log n)$$

Q8. What condition is required for Binary Search?

Answer:

The data must be **sorted** before Binary Search can be directly applied.

AOA_Teaching_Script

Q9. What is Strassen's Algorithm?

Answer:

Strassen's Algorithm is a Divide and Conquer algorithm for matrix multiplication. It reduces the number of recursive matrix multiplications from 8 to 7.

Its complexity is approximately:

$$O(n^{2.81})$$

instead of conventional:

$$O(n^3)$$

AOA_Teaching_Script

Part B – 5 Marks Questions with Answers

Q10. Explain Merge Sort.

Answer:

Merge Sort follows three steps:

Divide

Divide the array into two halves.

Conquer

Recursively sort both halves.

Combine

Merge the two sorted halves.

For example:

[38, 27, 43, 3]

is divided into:

[38, 27] [43, 3]

Then:

[27, 38] [3, 43]

Finally:

[3, 27, 38, 43]

Its recurrence is:

$$T(n) = 2T(n/2) + n$$

Using Master Method:

$$\Theta(n \log n)$$

AOA_Teaching_Script

Q11. Explain Quick Sort.

Answer:

Quick Sort selects a pivot and partitions the array around the pivot.

Steps:

1. Select pivot.
2. Partition array.
3. Place pivot in its correct position.
4. Recursively sort left partition.
5. Recursively sort right partition.

Average complexity:

$$O(n \log n)$$

Worst-case complexity:

$$O(n^2)$$

Randomized or median-of-three pivot selection can reduce the likelihood of poor partitions.

AOA_Teaching_Script

Q12. Explain Binary Search.

Answer:

Suppose:

$$A = [3, 7, 10, 14, 18, 23, 27, 31, 35]$$

To search for 23:

1. Find middle element.
2. Compare middle with 23.
3. If 23 is larger, search right half.
4. If smaller, search left half.
5. Repeat until found.

Each step eliminates approximately half of the remaining elements.

Therefore:

$$T(n) = O(\log n)$$

The notes use the "guess a number between 1 and 100" analogy to explain repeated halving.

AOA_Teaching_Script

UNIT III – GREEDY METHOD

Part A – 2 Marks Questions with Answers

Q1. Define Greedy Method.

Answer:

The Greedy Method builds a solution step-by-step by selecting the locally optimal choice at each stage.

However, a greedy choice does **not automatically guarantee** an optimal solution.

AOA_Teaching_Script

Q2. What is the Greedy-Choice Property?

Answer:

The greedy-choice property means that making a locally optimal choice at each stage can lead to a globally optimal solution.

Q3. What is Optimal Substructure?

Answer:

A problem has optimal substructure when an optimal solution contains optimal solutions to its subproblems.

Q4. Give an example where Greedy fails.

Answer:

Consider coins:

$$\{1, 3, 4\}$$

Target:

$$6$$

Greedy chooses:

$$4 + 1 + 1 = 6$$

using 3 coins.

But the optimal solution is:

$$3 + 3 = 6$$

using only 2 coins.

Therefore, Greedy does not always produce an optimal solution.

AOA_Teaching_Script

Q5. What is Dijkstra's Algorithm?

Answer:

Dijkstra's Algorithm finds shortest paths from a single source vertex to all other vertices in a weighted graph with appropriate non-negative edge weights.

Q6. Why does Dijkstra's Algorithm not work with negative weights?

Answer:

Dijkstra finalizes the currently smallest tentative distance. A later negative edge could produce a shorter path to an already finalized vertex, invalidating the algorithm's assumption.

AOA_Teaching_Script

Q7. What is Fractional Knapsack?

Answer:

Fractional Knapsack is an optimization problem in which items can be divided into fractions.

The greedy strategy selects items according to:

$$\frac{\text{Value}}{\text{Weight}}$$

in descending order.

AOA_Teaching_Script

Q8. What is an MST?

Answer:

A Minimum Spanning Tree is a spanning tree of a connected weighted graph having minimum possible total edge weight.

A spanning tree containing V vertices has:

$$V - 1$$

edges.

Q9. What is Kruskal's Algorithm?

Answer:

Kruskal's Algorithm constructs an MST by:

1. Sorting edges by increasing weight.
2. Selecting the smallest edge.
3. Rejecting an edge if it forms a cycle.
4. Continuing until $V - 1$ edges are selected.

AOA_Teaching_Script

Q10. What is Prim's Algorithm?

Answer:

Prim's Algorithm starts from any vertex and repeatedly selects the cheapest edge connecting the current tree to an unconnected vertex.

AOA_Teaching_Script

Part B – 5 Marks Questions with Answers

Q11. Explain the Greedy Method.

Answer:

The Greedy Method constructs a solution incrementally.

At every step:

1. Consider available choices.
2. Select the locally best choice.
3. Add it to the solution.
4. Continue until a complete solution is obtained.

Two important properties are:

1. Greedy-Choice Property

A locally optimal choice can lead to a globally optimal solution.

2. Optimal Substructure

An optimal solution contains optimal solutions to its subproblems.

Examples:

- Dijkstra's Algorithm
- Fractional Knapsack
- Kruskal's Algorithm
- Prim's Algorithm

The method must be justified for each specific problem rather than assumed to be correct automatically.

AOA_Teaching_Script

Q12. Solve Fractional Knapsack.

Given:

Item	Weight	Value	Value/Weight
I1	10	60	6
I2	20	100	5
I3	30	120	4

Capacity:

$$W = 50$$

Step 1: Select I1

Weight = 10

Value = 60

Remaining capacity:

$$50 - 10 = 40$$

Step 2: Select I2

Weight = 20

Value = 100

Remaining capacity:

$$40 - 20 = 20$$

Step 3: Take fraction of I3

I3 weighs 30, but only 20 units can be taken.

Fraction:

$$\frac{20}{30} = \frac{2}{3}$$

Value obtained:

$$120 \times \frac{2}{3} = 80$$

Total Value

$$60 + 100 + 80 = 240$$

Therefore:

$$\text{Maximum Value} = 240$$

This is the same worked example structure given in the notes.

AOA_Teaching_Script

Q13. Explain Dijkstra's Algorithm.

Answer:

Dijkstra's Algorithm maintains a tentative distance for every vertex.

Steps

1. Set source distance to 0.
2. Set all other distances to infinity.
3. Select the unvisited vertex having the smallest tentative distance.
4. Mark it as finalized.
5. Relax all its adjacent edges.
6. Repeat until all required vertices are finalized.

Relaxation

For an edge $u \rightarrow v$:

$$\text{if } d[u] + w(u, v) < d[v]$$

then:

$$d[v] = d[u] + w(u, v)$$

The algorithm relies on the fact that finalized distances cannot later become smaller, which is why negative edge weights cause problems.

AOA_Teaching_Script

Q14. Explain Kruskal's Algorithm.

Answer:

Kruskal's Algorithm finds an MST by selecting edges in increasing order of weight.

Steps

1. List all edges.
2. Sort edges in ascending order.
3. Select the smallest edge.

4. Check whether it creates a cycle.
5. If no cycle is formed, include it.
6. If it forms a cycle, reject it.
7. Continue until $V - 1$ edges are selected.

Example

Suppose the sorted edges are:

$$AB(1), BC(2), AC(3), BD(4), CD(5)$$

Select:

$$AB$$

Select:

$$BC$$

Reject:

$$AC$$

because it creates a cycle $A - B - C - A$.

Select:

$$BD$$

The MST is:

$$\{AB, BC, BD\}$$

Total cost:

$$1 + 2 + 4 = 7$$

The notes emphasize sorting edges and rejecting edges that create cycles.

AOA_Teaching_Script

Q15. Explain Prim's Algorithm.

Answer:

Prim's Algorithm grows an MST from a selected starting vertex.

Steps

1. Select any starting vertex.
2. Mark it as part of the tree.
3. Find the minimum-weight edge connecting the tree to an unvisited vertex.
4. Add that edge and vertex.
5. Repeat until all vertices are included.

Unlike Kruskal, Prim's algorithm grows **one connected tree** from the starting vertex.

AOA_Teaching_Script

Q16. Compare Kruskal's and Prim's Algorithms.

Feature	Kruskal	Prim
Basic approach	Edge-based	Vertex/tree-based
Starting point	Not required	Required
Edge processing	Globally smallest edge	Cheapest edge from current tree
Cycle handling	Explicitly checked	Naturally avoided when adding new vertex
Useful for	Sparse graphs	Often useful for dense graphs
Main data structure	Union-Find	Priority Queue/Heap
Result	MST	MST

The notes specifically indicate that Prim's with a heap is often advantageous for dense graphs, while Kruskal is convenient for sparse graphs or already-sorted edges.

AOA_Teaching_Script

Important 10-Mark Answers

Q17. Explain Fractional Knapsack using Greedy Method.

Answer:

The Fractional Knapsack problem consists of n items where each item has:

- Weight w_i
- Value v_i

The knapsack has capacity W .

Unlike 0/1 Knapsack, an item can be divided.

Greedy Criterion

Calculate:

$$R_i = \frac{v_i}{w_i}$$

Sort all items by decreasing R_i .

Then:

1. Take the item with the highest ratio.
2. If the entire item fits, take it completely.
3. Otherwise, take the required fraction.
4. Stop when capacity is full.

Complexity

Sorting requires:

$$O(n \log n)$$

The selection process requires:

$$O(n)$$

Therefore:

$$O(n \log n)$$

The greedy strategy works because replacing a lower value/weight item with a higher value/weight item cannot reduce the total value.

AOA_Teaching_Script

Q18. Explain Kruskal's Algorithm with an example.

Answer:

Kruskal's algorithm is a greedy algorithm for finding a Minimum Spanning Tree.

Algorithm

Kruskal (G)

Sort all edges in increasing order of weight

```

MST = empty

for each edge (u,v) in sorted order
    if adding (u,v) does not form a cycle
        add (u,v) to MST

    if MST contains V-1 edges
        stop

return MST

```

Example

Consider edges:

Edge	Weight
AB	1
BC	2
AC	3
BD	4
CD	5

Sorted order:

AB, BC, AC, BD, CD

Select AB.

Select BC.

AC is rejected because:

$A - B - C - A$

forms a cycle.

Select BD.

Now $V - 1 = 3$ edges have been selected.

MST:

$\{AB, BC, BD\}$

Total cost:

7

Q19. Explain Prim's Algorithm with an example.

Answer:

Prim's algorithm begins with any vertex and expands the tree by selecting the cheapest edge connecting the current tree to an outside vertex.

Suppose we start at A .

At every stage:

Select minimum edge crossing from tree to unvisited vertex

For example, if:

$$AB = 1, \quad AC = 3, \quad BC = 2, \quad BD = 4$$

Start with A .

Select:

$$AB = 1$$

Tree:

$$\{A, B\}$$

Next cheapest edge connecting the tree to an outside vertex:

$$BC = 2$$

Tree:

$$\{A, B, C\}$$

Next:

$$BD = 4$$

Final MST:

$$\{AB, BC, BD\}$$

Total cost:

$$1 + 2 + 4 = 7$$

Most Important Numerical Answers

1. Selection Sort

For $n = 10$:

$$\begin{aligned}\text{Comparisons} &= \frac{n(n-1)}{2} \\ &= \frac{10(9)}{2} \\ &= \boxed{45}\end{aligned}$$

2. Binary Search

For $n = 100$, approximately:

$$\log_2 100 \approx 6.64$$

Therefore, roughly:

7 comparisons

The key idea is repeated halving of the search space.

AOA_Teaching_Script

3. Selection Sort Trace

Input:

[29, 10, 14, 37, 13]

Pass 1:

[10, 29, 14, 37, 13]

Pass 2:

[10, 13, 14, 37, 29]

Pass 3:

[10, 13, 14, 37, 29]

Pass 4:

[10, 13, 14, 29, 37]

4. Insertion Sort Trace

Input:

[37, 29, 14, 13, 10]

After 29:

[29, 37, 14, 13, 10]

After 14:

[14, 29, 37, 13, 10]

After 13:

[13, 14, 29, 37, 10]

After 10:

[10, 13, 14, 29, 37]

The notes specifically contrast nearly sorted and reverse-sorted inputs to demonstrate Insertion Sort's adaptive behavior.

AOA_Teaching_Script

Quick Revision Table

Topic	Important Result
Time Complexity	Growth of computational work
Space Complexity	Growth of memory requirement
Big-O	Upper bound
Ω	Lower bound
Θ	Tight bound
Selection Sort	$\Theta(n^2)$

Topic	Important Result
Insertion Sort Best	$\Theta(n)$
Insertion Sort Worst	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$
Quick Sort Average	$O(n \log n)$
Quick Sort Worst	$O(n^2)$
Binary Search	$O(\log n)$
Strassen	Approximately $O(n^{2.81})$
Dijkstra	Single-source shortest path
Fractional Knapsack	Greedy by value/weight
Kruskal	Edge-based MST
Prim	Vertex/tree-based MST
MST edges	$V - 1$
Greedy	Local optimum at each step

Suggested Examination Pattern

For a **60/70-mark university paper**, these three units can be converted into a balanced paper such as:

Section A – Short Answer

10 × 2 = 20 marks

- Complexity
- Big-O/ Ω / Θ
- Master Method
- Selection/Insertion Sort
- Divide and Conquer
- Merge/Quick Sort
- Binary Search

- Greedy Method
- Dijkstra
- Kruskal/Prim

Section B – Descriptive

5 × 5 = 25 marks

Questions on:

- Master Method
- Selection/Insertion Sort
- Merge Sort
- Quick Sort
- Fractional Knapsack
- Dijkstra
- Kruskal/Prim

Section C – Problem Solving

3 × 10 = 30 marks

1. Solve a recurrence using Master Method.
2. Trace Merge Sort/Quick Sort/Binary Search.
3. Solve Dijkstra/Fractional Knapsack/MST.

This structure keeps the questions closely aligned with the **teaching notes' worked examples, anticipated doubts, and algorithm-analysis emphasis** rather than introducing unrelated topics.

AOA_Teaching_Script +2

Sources